6CCS3ML1 (Machine Learning)

Coursework 2

(Version 1.7)

1 Overview

For this coursework, you will have to implement the Q-learning algorithm. Your code will again be controlling Pacman, in the classic game, and the Q-learning algorithm will be helping Pacman choose how to move. Your Q-learning algorithm should be able, once it has done its learning, to play a pretty good game, and its ability to play a good game will be part of what we assess.

This coursework is worth 10% of the marks for the module.

Note: Failure to follow submission instructions will result in a deduction of 10% of the marks you earn for this coursework.

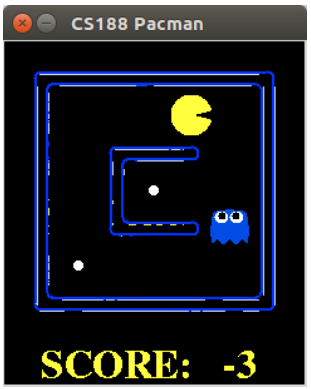

Figure 1: The `smallGrid` version of Pacman.

2 Getting started

2.1 Start with Pacman

The Pacman code that we will be using for the 6CCS3ML1 coursework was originally developed at UC Berkeley for their AI course. Note that we will **not** be doing any of their projects.

You should:

1. Download `pacman-cw2.zip` from KEATS.
2. Save that file to your account at KCL (or to your own computer) in a new folder.
3. Unzip the archive. This will create a folder `pacman-cw2`

4. From the command line (you will need to use the command line in order to use the various options), switch to the folder `pacman-cw2`.

5. Now type:

   ```
   python3 pacman.py -p RandomAgent -n 10 -l smallGrid
   ```

   and watch it run.

This command illustrates a few aspects of the Pacman code that you will need to understand:

- `-n 10` runs the game 10 times.
- `-l smallGrid` runs the very reduced game you see in Figure 1.

  This is not a very interesting game for a human to play, but it is moderately challenging for a reinforcement learning program to learn to play.
- `-p RandomAgent` tells the `pacman.py` code to let Pacman be controlled by an object that is an instance of a class called `RandomAgent`.

  The program then searches through all files with a name that **ends** in:

  `Agents.py`

  looking for this class. If the class isn't in an appropriately named file, you will get the error:

  ```
  Traceback (most recent call last):
    File "pacman.py", line 679, in <module>
      args = readCommand( sys.argv[1:] ) # Get game components based on input
    File "pacman.py", line 541, in readCommand
      pacmanType = loadAgent(options.pacman, noKeyboard)
    File "pacman.py", line 608, in loadAgent
      raise Exception('The agent ' + pacman + ' is not specified in any *Agents.py.')
  ```

  In this case the class is found in `sampleAgents.py`

`RandomAgent` just picks actions at random. When there is no ghost, it will win a game eventually by eating all the food[1], but when a ghost is present, `RandomAgent` dies pretty quickly.

2.2 Towards an RL Pacman

Your task is to write code to learn how to play this small version of Pacman. To get you started, we have provided a skeleton of code that will do this, found in:

`mlLearningAgents.py`

This file contains a class `QLearnAgent`, and that class includes several methods.

`init()` This is the constructor for `QLearnAgent`. It is called by the game when the game starts up (because the game starts up the learner).

The version of `init()` in `QLearnAgent` allows you to pass parameters from the command line. Some of these you know from the lectures:

[1] If you want to see this happen, run `python3 pacman.py -p RandomAgent -k 0 -l smallGrid`. The flag `-k` sets the number of ghosts.

- `alpha`, the learning rate
- `gamma`, the discount rate
- `epsilon`, the exploration rate

and you will use them in your implementation. The other:

- `numTraining`

allows you to run some games as training episodes and some as real games.

All the constructor does is take these values from the command line, if you pass them, and write them into sensibly named variables. If you don't pass values from the command line, they take the default values you see in the code. These values work perfectly well, but if you want to play with different values, then you can do that like this:

```
python3 pacman.py -p QLearnAgent -l smallGrid -a numTraining=2 -a alpha=0.2
```

Note that you need to have no space between parameter and value `alpha=0.2`, and you need a separate `-a` for each one.

`getAction()` This function is called by the game every time that it wants Pacman to make a move (which is every step of the game).

This is the core of code that controls Pacman. It has access to information about the position of Pacman, the position of ghosts, the location of food, and the score. The code shows how to access all of these, and if you run the code (as above) all these values are printed out each time `getAction()` is called.

The only bit that maybe needs some explanation is the food. Food is represented by a grid of letters, one for each square in the game grid. If the letter is `F`, there is no food in that square. If the letter is `T`, then there is food in that square. (Walls are not represented).

The main job of `getAction()` is to decide what move Pacman should make, so it has to return an action. The current code shows how to do that but just makes the choice randomly.

`final()` This function is called by the game when a Pacman has been killed by a ghost, or when Pacman eats the last food (and so wins).

Right now all this function does is to keep track of the number of episodes (more on what they are below) and sets `epsilon` and `alpha` to zero when a certain number have happened (more on that in a minute also).

3 What you have to do (and what you aren't allowed to do)

3.1 Implementation

You should fill out `QLearnAgent` with code that performs Q-learning. You should implement your code in the `mlLearningAgents.py` file.

You will be assessed on your implementation of Q-learning by **the functionality of several methods defined in the file**. These are clearly indicated in the file.

Because of the way that Pacman works, your learner needs to include two separate pieces:

1. Something that learns, adjusting Q-values based on how well the learner plays the game.

2. Something that chooses how to act. This decision can be based on the Q-values, but also has to make sure that the learner does enough exploring.

You need to do that because using Pacman forces you to do online learning. The only way that Pacman learns is by actually playing in a game. Initially it will play badly, but over time it should get better and better. However, it will learn slowly. Indeed, it will need to play around 2000 games to get to be any good.

See Section 4 for a description of what to submit. Note that we will evaluate your code by running the command:

```
python3 pacman.py -p QLearnAgent -x 2000 -n 2010 -l smallGrid
```

so we will train the learner for 2000 episodes and then run it for 10 non-training episodes (see below). We will use the score that your learner gets in those 10 episodes to decide how good your solution is. To get good marks, your `QLearnAgent` needs to win around 8 of the games (which is very achievable).

3.2 Things to know

- The code has some support for learning. First, you can run several games one after another by using the `-n` parameter. This:

  ```
  python3 pacman.py -p QLearnAgent -l smallGrid -n 5
  ```

  will run the same game 5 times. Each of these runs is one of the *episodes* that were mentioned in the text about `final()`. The really useful bit is that the same instance of `QLearnAgent` gets run all five times, so that it can learn over all five episodes.

  Even more helpful, you can do some of the runs without the graphical interface, so that they run quickly. If you run:

  ```
  python3 pacman.py -p QLearnAgent -l smallGrid -x 5 -n 10
  ```

  it will do the first 5 runs without the interface. The remainder of the runs will happen with the interface. During the training runs, the system does not record the scores, so the average score (and results) printed at the end do not include the results of the training episodes.

  The command-line switch `-numTraining` is just a synonym for `-x`, so the value you specify with `-x` is the value that ends up in the variable `numTraining` in the code. This is why `final()` counts the number of episodes and compares it to `numTraining` — that is how the agent knows when training is over and it is showtime. We set `epsilon` and `alpha` to zero at that point because (as you know from the lecture) these are parameters that control learning. An ϵ-greedy learner chooses not to do the best action that it knows with a probability ϵ, so if we set ϵ to zero the learner will always do what it thinks is best (and it won't get killed because it does something random).[2] Similarly, if you set α to zero in a Q-learning learner, then the update doesn't change the Q-values.

- Though, as the lecture says, in theory you need to adjust the learning rate over time, I did not find that was necessary.[3]

[2] It might still make a bad move, but it won't know it is a bad move.

[3] It is quite possible that the values did not converge as a result, or that they did not converge on the optimal values, but the learner could play a good game, and that is what matters.

- There is no way to access the reward for winning (or the cost of losing) outside `final()`. (`getAction()` is not called once the episode is over.) You will need those rewards to learn how to avoid the ghost and how to win the game.[4]

3.3 Limitations

There are some limitations on what you can submit.

1. Your code should be in Python 3.
2. Your code will be tested in the same environment as we have been using in the lab (the standard Anaconda Python 3 distribution). If your code does not run when we test it, you will lose marks.
3. You are not allowed to modify any of the files in `pacman-cw2.zip`.

   The only thing you can do as part of the coursework is to write code in `mlLearningAgents.py`.
4. You are not allowed to copy, without credit, code that you might get from other students or find lying around on the Internet. (This includes the use of code that was distributed as part of the module — if you use code from files other than `mlLearningAgents.py` without attribution, we will consider that to be plagiarism.) We will be checking.

   This is the usual plagiarism statement. When you submit work to be marked, you should only seek to get credit for work you have done yourself.

   When the work you are submitting is code, you can use code that other people wrote, but you have to say clearly that another person wrote it (you do that by putting in a comment that says who wrote it). That way we can adjust your mark to take account of the work that you didn't do.

4 What you have to hand in

Your submission should consist of a single ZIP file (KEATS will be configured to only accept a single file.) This ZIP file must include a single Python file (`mlLearningAgents.py`).

The ZIP file must be named[5]:

```
cw2-<lastname>-<firstname>.zip
```

so my ZIP file would be named `cw2-cocarascu-oana.zip`. Submissions that are not in zip format will lose marks.

Remember that we are going to evaluate your code by running your code using variations on

```
python3 pacman.py -p QLearnAgent -x 2000 -n 2010 -l smallGrid
```

and we will do this in a vanilla copy of the `pacman-cw2` folder, so the base class for your agent must be called `QLearnAgent`.

[4] Without those rewards, the learner will choose to run towards the ghost to end the game quickly and minimise its losses — very rational when you don't get the big positive reward for winning.

[5] The person who submits for the group should name the file.

Do not just include the whole `pacman-cw2` folder. You should only include the one file that includes the code you have written.

Submissions that do not follow these instructions will lose marks.

5 How your work will be marked

There will be six components of the mark for your work, four concerned with functionality and two with form.

1. Functionality

   We will test your code by running your `.py` file against a clean copy of `pacman-cw2`.

   (a) For full marks for functionality, your code is required to run when we invoke the command:

   ```
   python3 pacman.py -p QLearnAgent -x 2000 -n 2010 -l smallGrid
   ```

   If your code does not run, you will lose marks.

   (b) We will also look at your code for evidence of the use of Q-learning. Code that does not use Q-learning will lose marks.

   (c) When we run your code using:

   ```
   python3 pacman.py -p QLearnAgent -x 2000 -n 2010 -l smallGrid
   ```

   it is required to win 8 of 10 games. (If it fails to do this on the first run, we will run it again and return the maximum number of wins from the two runs). If your code wins less than 8 games, you will get a mark that reflects how many games your code wins — more wins equals more marks.

   Note that we will limit how long your code has to demonstrate that it can win 8 games out of 10. We will terminate the run after 5 minutes. If your code has won less than 8 games within these 5 minutes, you will get a mark that reflects how many games your code has won — more wins equals more marks.

   (d) Finally, to test generalisability, we will run your code on another grid. Code that fails to run on another grid will lose marks. We will not test the performance of your code, that is how many games it wins, we will only test if it runs.

2. Form

   (a) There are no particular requirements on the way that your code is structured, but it should follow standard good practice in software development and will be marked accordingly.

   (b) All good code is well documented, and your work will be partly assessed by the comments you provide in your code. If we cannot understand from the comments what your code does, then you will lose marks.

A copy of the marksheet, which shows the distribution of marks across the different elements of the coursework, is available on KEATS, together with FAQs.